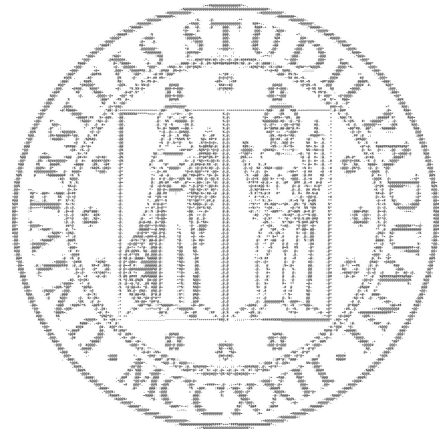


Mitigating Use-After-Free Vulnerabilities in Modern Operating Systems and Browsers: A Comprehensive Survey

COMPUTERS AND NETWORKS SECURITY 2024-2025

Instructor: Mauro Migliardi



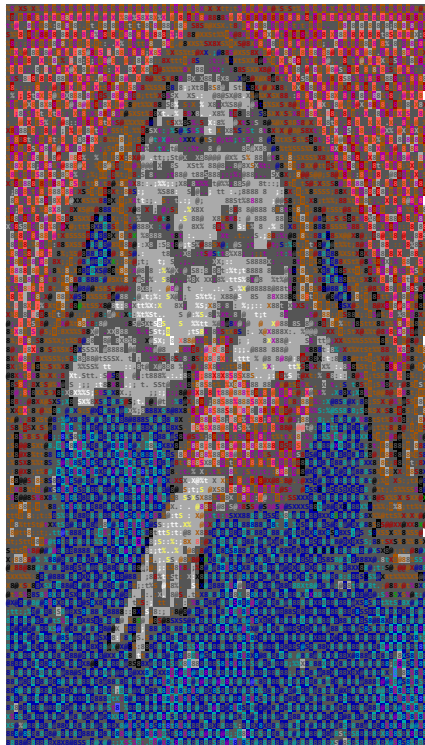
Fatemeh Mahvari — University of Padua

AGENDA

- Introduction & problem scope
- OS-level UAF defenses
- Browser-level mitigations
- Allocator internals (BIBOP, canaries, RIO, type-safe reuse)
- Emerging directions (tagging, capabilities, isolation)
- Case studies
- Trade-offs & effectiveness
- Conclusions & future work

INTRODUCTION

What Is a Use-After-Free (UAF)?



Program frees an object but later uses a stale pointer to it. This **dangling pointer** may now reference reused memory.

- Leads to arbitrary read/write, RCE, or data leaks in C/C++
- 80% of UAFs rated high severity (2007–2016 study)

```

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    /* 1. allocate two uint32_t words (8 bytes) */
    uint32_t *p = malloc(2 * sizeof(uint32_t));

    /* 2. create a second pointer that refers to the 2nd word */
    uint32_t *q = p + 1;          // <-- alias to the same block

    /* -- program logic . . . ----- */

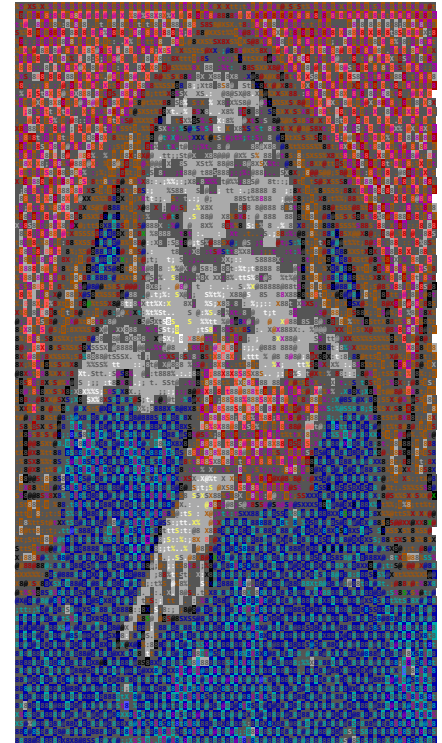
    /* 3. free the original allocation - p and q become dangling */
    free(p);

    /* 4. attacker (or benign code) forces a new allocation of
       the same size. Very often, the allocator will hand us
       the *same* address. */
    char *u = malloc(2 * sizeof(uint32_t));

    /* 5. stale pointer q still "looks" valid and now corrupts u */
    *q = 0xDEADBEEF;              // <== Use-After-Free write

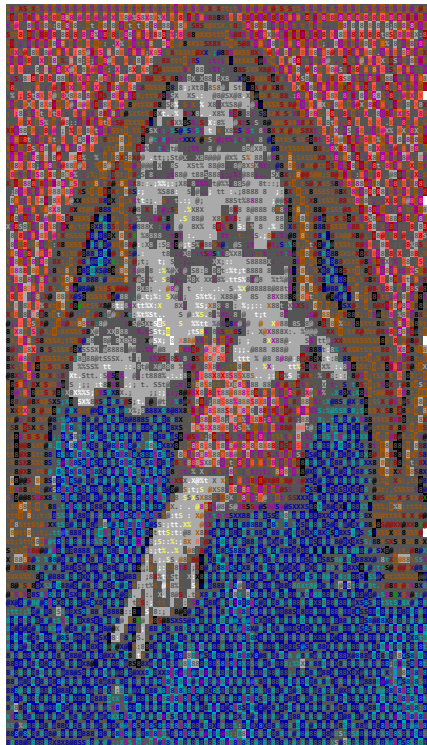
    printf("u[0..3] now contains 0x%X\n", *(uint32_t *)u);
    free(u);
    return 0;
}

```



Minimal C example of a Use-After-Free vulnerability (adapted from Listing 1 in Bernhard et al.'s xTag paper).

Why UAFs Matter (browsers & kernels)



Large C/C++ codebases → manual memory pitfalls

- Chromium: UAF = largest single bug class among high/critical issues
- Firefox: a large share of CVEs historically due to UAF

High privileges + huge attack surface → high impact



OPERATING SYSTEM-LEVEL DEFENSES AGAINST UAF

Heap Randomization & Quarantine

- Random free-chunk selection (e.g., Windows LFH)
 - Address obfuscation + random gaps between allocations
- Use of **Quarantine** buffer: delays reuse (e.g., Scudo in Android)
 - ensure the dangling pointer's memory is not immediately recycled for attacker-controlled data
 - More entropy → less deterministic reuse (but still probabilistic)

[illegible]

Isolated Heap Regions per Allocation

- **complete partitioning:**
 - isolating objects at page granularity
 - constraining reuse within type-/cache-specific pools

- **Slab architecture** in Linux (What kernels already do)

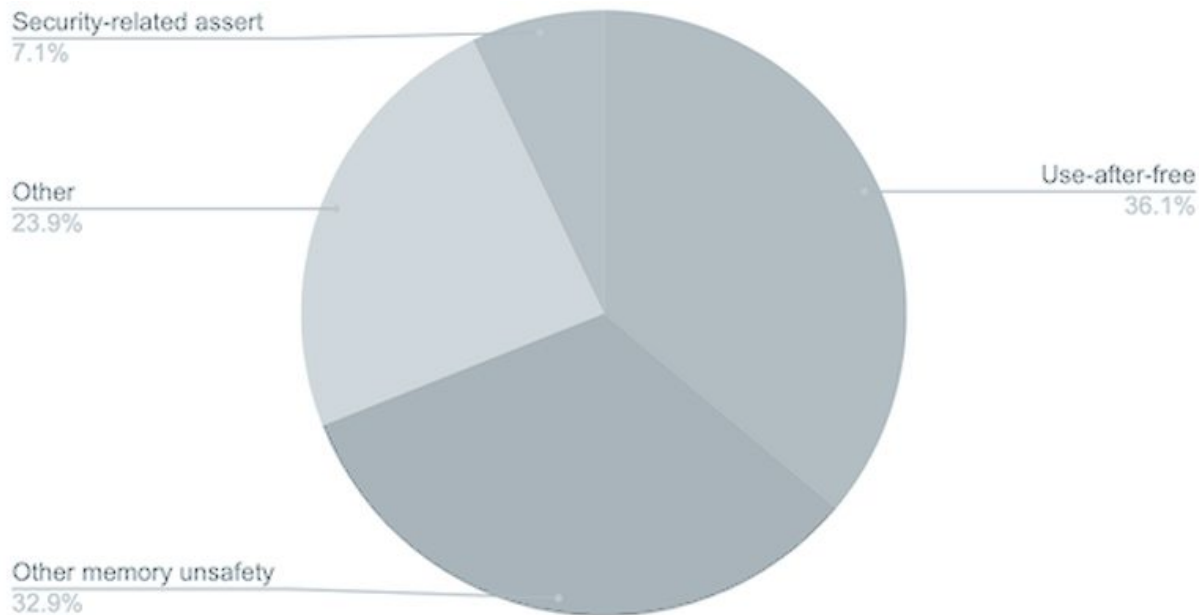
- Protection keys

```

1:00000000
2:
3:
4:
5:
6:
7:
8:
9:
10:
11:
12:
13:
14:
15:
16:
17:
18:
19:
20:
21:
22:
23:
24:
25:
26:
27:
28:
29:
30:
31:
32:
33:
34:
35:
36:
37:
38:
39:
40:
41:
42:
43:
44:
45:
46:
47:
48:
49:
50:
51:
52:
53:
54:
55:
56:
57:
58:
59:
60:
61:
62:
63:
64:
65:
66:
67:
68:
69:
70:
71:
72:
73:
74:
75:
76:
77:
78:
79:
80:
81:
82:
83:
84:
85:
86:
87:
88:
89:
90:
91:
92:
93:
94:
95:
96:
97:
98:
99:
100:
101:
102:
103:
104:
105:
106:
107:
108:
109:
110:
111:
112:
113:
114:
115:
116:
117:
118:
119:
120:
121:
122:
123:
124:
125:
126:
127:
128:
129:
130:
131:
132:
133:
134:
135:
136:
137:
138:
139:
140:
141:
142:
143:
144:
145:
146:
147:
148:
149:
150:
151:
152:
153:
154:
155:
156:
157:
158:
159:
160:
161:
162:
163:
164:
165:
166:
167:
168:
169:
170:
171:
172:
173:
174:
175:
176:
177:
178:
179:
180:
181:
182:
183:
184:
185:
186:
187:
188:
189:
190:
191:
192:
193:
194:
195:
196:
197:
198:
199:
200:
201:
202:
203:
204:
205:
206:
207:
208:
209:
210:
211:
212:
213:
214:
215:
216:
217:
218:
219:
220:
221:
222:
223:
224:
225:
226:
227:
228:
229:
230:
231:
232:
233:
234:
235:
236:
237:
238:
239:
240:
241:
242:
243:
244:
245:
246:
247:
248:
249:
250:
251:
252:
253:
254:
255:
256:
257:
258:
259:
260:
261:
262:
263:
264:
265:
266:
267:
268:
269:
270:
271:
272:
273:
274:
275:
276:
277:
278:
279:
280:
281:
282:
283:
284:
285:
286:
287:
288:
289:
290:
291:
292:
293:
294:
295:
296:
297:
298:
299:
300:
301:
302:
303:
304:
305:
306:
307:
308:
309:
310:
311:
312:
313:
314:
315:
316:
317:
318:
319:
320:
321:
322:
323:
324:
325:
326:
327:
328:
329:
330:
331:
332:
333:
334:
335:
336:
337:
338:
339:
340:
341:
342:
343:
344:
345:
346:
347:
348:
349:
350:
351:
352:
353:
354:
355:
356:
357:
358:
359:
360:
361:
362:
363:
364:
365:
366:
367:
368:
369:
370:
371:
372:
373:
374:
375:
376:
377:
378:
379:
380:
381:
382:
383:
384:
385:
386:
387:
388:
389:
390:
391:
392:
393:
394:
395:
396:
397:
398:
399:
400:
401:
402:
403:
404:
405:
406:
407:
408:
409:
410:
411:
412:
413:
414:
415:
416:
417:
418:
419:
420:
421:
422:
423:
424:
425:
426:
427:
428:
429:
430:
431:
432:
433:
434:
435:
436:
437:
438:
439:
440:
441:
442:
443:
444:
445:
446:
447:
448:
449:
450:
451:
452:
453:
454:
455:
456:
457:
458:
459:
460:
461:
462:
463:
464:
465:
466:
467:
468:
469:
470:
471:
472:
473:
474:
475:
476:
477:
478:
479:
480:
481:
482:
483:
484:
485:
486:
487:
488:
489:
490:
491:
492:
493:
494:
495:
496:
497:
498:
499:
500:
501:
502:
503:
504:
505:
506:
507:
508:
509:
510:
511:
512:
513:
514:
515:
516:
517:
518:
519:
520:
521:
522:
523:
524:
525:
526:
527:
528:
529:
530:
531:
532:
533:
534:
535:
536:
537:
538:
539:
540:
541:
542:
543:
544:
545:
546:
547:
548:
549:
550:
551:
552:
553:
554:
555:
556:
557:
558:
559:
560:
561:
562:
563:
564:
565:
566:
567:
568:
569:
570:
571:
572:
573:
574:
575:
576:
577:
578:
579:
580:
581:
582:
583:
584:
585:
586:
587:
588:
589:
590:
591:
592:
593:
594:
595:
596:
597:
598:
599:
600:
601:
602:
603:
604:
605:
606:
607:
608:
609:
610:
611:
612:
613:
614:
615:
616:
617:
618:
619:
620:
621:
622:
623:
624:
625:
626:
627:
628:
629:
630:
631:
632:
633:
634:
635:
636:
637:
638:
639:
640:
641:
642:
643:
644:
645:
646:
647:
648:
649:
650:
651:
652:
653:
654:
655:
656:
657:
658:
659:
660:
661:
662:
663:
664:
665:
666:
667:
668:
669:
670:
671:
672:
673:
674:
675:
676:
677:
678:
679:
680:
681:
682:
683:
684:
685:
686:
687:
688:
689:
690:
691:
692:
693:
694:
695:
696:
697:
698:
699:
700:
701:
702:
703:
704:
705:
706:
707:
708:
709:
710:
711:
712:
713:
714:
715:
716:
717:
718:
719:
720:
721:
722:
723:
724:
725:
726:
727:
728:
729:
730:
731:
732:
733:
734:
735:
736:
737:
738:
739:
740:
741:
742:
743:
744:
745:
746:
747:
748:
749:
750:
751:
752:
753:
754:
755:
756:
757:
758:
759:
760:
761:
762:
763:
764:
765:
766:
767:
768:
769:
770:
771:
772:
773:
774:
775:
776:
777:
778:
779:
780:
781:
782:
783:
784:
785:
786:
787:
788:
789:
790:
791:
792:
793:
794:
795:
796:
797:
798:
799:
800:
801:
802:
803:
804:
805:
806:
807:
808:
809:
810:
811:
812:
813:
814:
815:
816:
817:
818:
819:
820:
821:
822:
823:
824:
825:
826:
827:
828:
829:
830:
831:
832:
833:
834:
835:
836:
837:
838:
839
```

BROWSER-LEVEL UAF MITIGATIONS

Distribution of High-Severity Chrome Security Bugs



memory unsafety dominates and use-after-free is the largest single category.

Source: The Chromium Projects

Chrome's PartitionAlloc

- Partitions heap by **type** (DOM, rendering, JS buffers, misc) + **size classes**
- Prevents cross-type reuse from dangling pointers
- Out-of-line metadata + guard pages harden allocator
- Part of layered defense with site isolation & CFI

```
+-----+
|          GUARD PAGE (no access)          |
+-----+
```

```
+-----+
| PARTITION: DOM / Layout Engine Objects   |
| (Only DOM-ish objects live here; JS strings/buffers can't land inside) |
+-----+
```

```
[ Partition metadata (out-of-line; not inline with user objects) ]
```

```
+-----+
| SuperPage A (many OS pages)              |
+-----+
```

```
| Bucket: 64 B objects (size class)        |
| Slot Span #1 (N pages)                   |
+-----+
```

```
| A64 | F64 | A64 | F64 | F64 | A64 | ... |
+-----+
```

```
| ^   ^   ^                               |
| |   |   | +-- allocated slot (user object) |
| |   |   | +----- free slot (on per-bucket freelist) |
| +----- free slot                               |
+-----+
```

```
+-----+
| Bucket: 256 B objects                    |
| Slot Span #7 (N pages)                   |
+-----+
```

```
| A256 | A256 | F256 | F256 | A256 | F256 | ... |
+-----+
```

```
| freelist: F256 -> F256 -> F256 -> ... |
+-----+
```

```
... more buckets (512 B, 1 KB, 2 KB, ...) with their own
slot spans & freelists
```

Chrome's PartitionAlloc

Conceptual snapshot of how a PartitionAlloc-style heap can look.

It shows the main moving parts: type partitions, size classes (buckets), slot spans, slots, guard pages, and out-of-line metadata.

Chrome's PartitionAlloc

```
|
|
| [Guard Page] [SuperPage B] [Guard Page] ...
| (catches overflows between large regions)
+-----+
```

```
+-----+
|
|          GUARD PAGE (no access)
|
+-----+
```

```
+-----+
|
|    PARTITION: Rendering / Compositor Objects
| (same structure: buckets -> slot spans -> slots; separate from DOM)
|
+-----+
```

```
+-----+
|
|          GUARD PAGE (no access)
|
+-----+
```

```
+-----+
|
|    PARTITION: JS Buffers / ArrayBuffers
| (its own buckets and freelists; cannot mix with DOM partition)
|
+-----+
```

```
+-----+
|
|          GUARD PAGE (no access)
|
+-----+
```

```
+-----+
|
|    PARTITION: General / Misc Allocations
|
+-----+
```

Conceptual snapshot of how a PartitionAlloc-style heap can look.

It shows the main moving parts: type partitions, size classes (buckets), slot spans, slots, guard pages, and out-of-line metadata.

IE/Edge Isolated Heap & MemGC

- Most DOM objects are allocated from a dedicated heap
 - if an attacker holds a dangling pointer to a DOM object, they cannot fill that exact heap space with an arbitrary script
- **MemGC** does not immediately release freed objects: **Garbage Collection**

Firefox's Arena Allocators and Heap Hardening

- **Arenas** recycle fixed-size slots (some type/size segregation)
- **Frame poisoning** fills freed memory with invalid patterns to crash early
- Adoption of hardened allocators (e.g., Scudo on some platforms)
- **RLBox** sandboxing to contain third-party library bugs
- Adopting **Rust** for new components: Project “**Oxidization**”

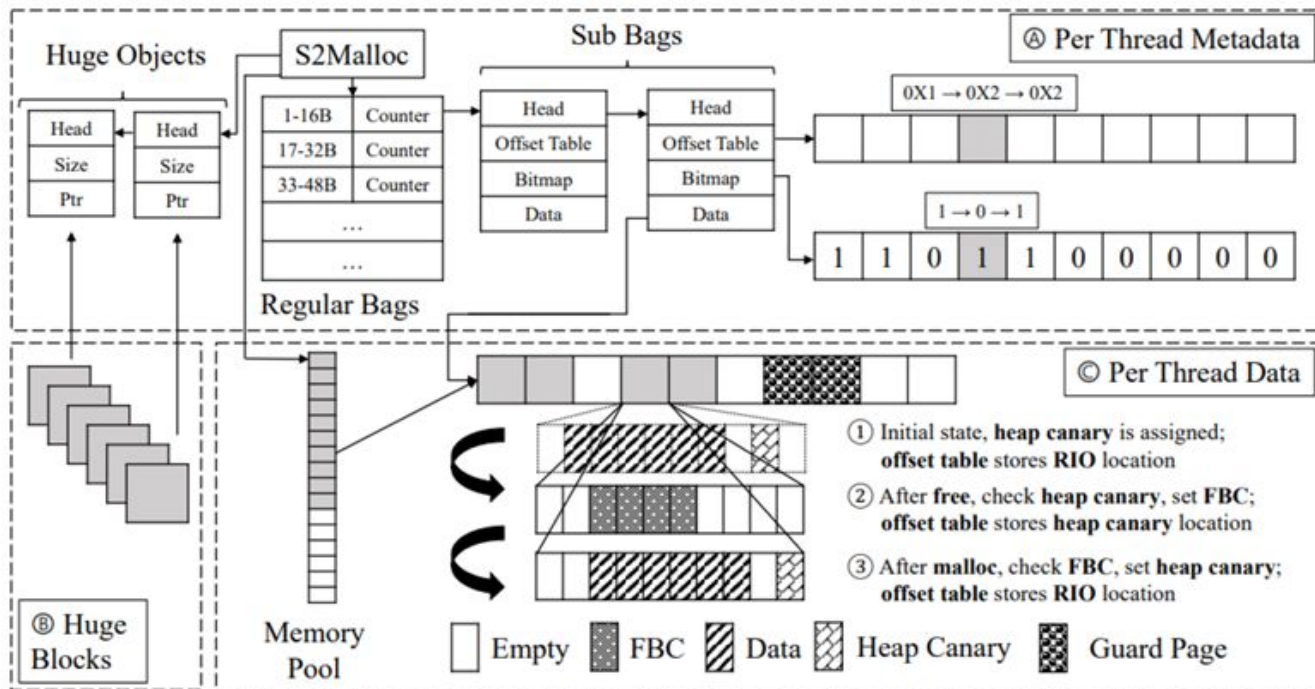
TECHNICAL MITIGATION MECHANISMS IN MEMORY ALLOCATORS

BIBOP Layout

- Separate **bags** per size class; fixed-size **slots** tracked by bitmaps
- Out-of-line metadata prevents metadata corruption
- Natural containment: UAF stays within size-segregated regions
- Guard pages can catch OOB or UAF derefs

Allocators: DieHarder, Guarder, SlimGuard, S2malloc

BIBOP Layout



S2malloc's combo: quarantine + RIO + FBC

Free-Block Canary

- On **free**, write random canary inside the slot
- On **reuse**, verify canary → detect tampering; abort or alert
- Strong against writes; reads-after-free still possible
- Random canary **offset** makes bypass harder

Allocators: S2malloc

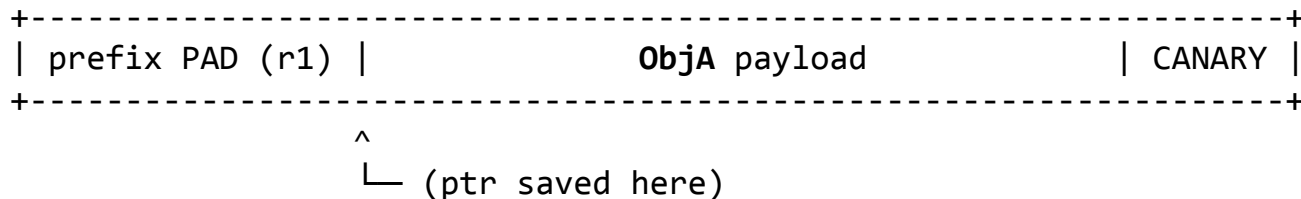
Randomized In-Block Offset (RIO)

- Add random prefix padding so object start **shifts** each time
- Breaks attacker's field-offset assumptions when slot is reused
- Often pairs well with canaries & quarantine
- Security vs memory overhead trade-off

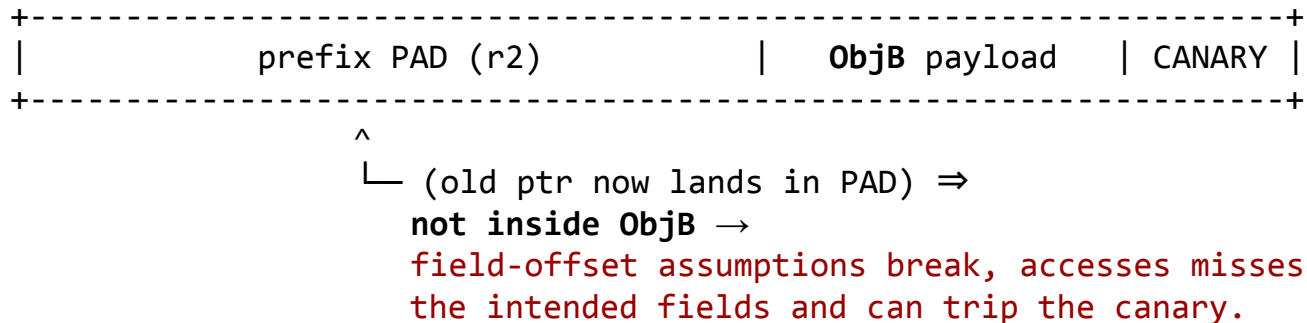
Allocators: S2malloc, ZEUS, Guarder

Randomized In-Block Offset (RIO)

1) First allocation



2) Second allocation (different random offset)



Type-Safe Reuse

- Reuse freed memory **only** for same **type** (or same **call site**)
- Blocks cross-type UAFs (e.g., file object → string corruption)
- Doesn't stop **same-type** UAFs; metadata/complexity overheads
- Designed to integrate with large codebases (e.g., Firefox allocators)

Allocators: Cling, Type-After-Type

EMERGING DIRECTIONS IN UAF MITIGATIONS

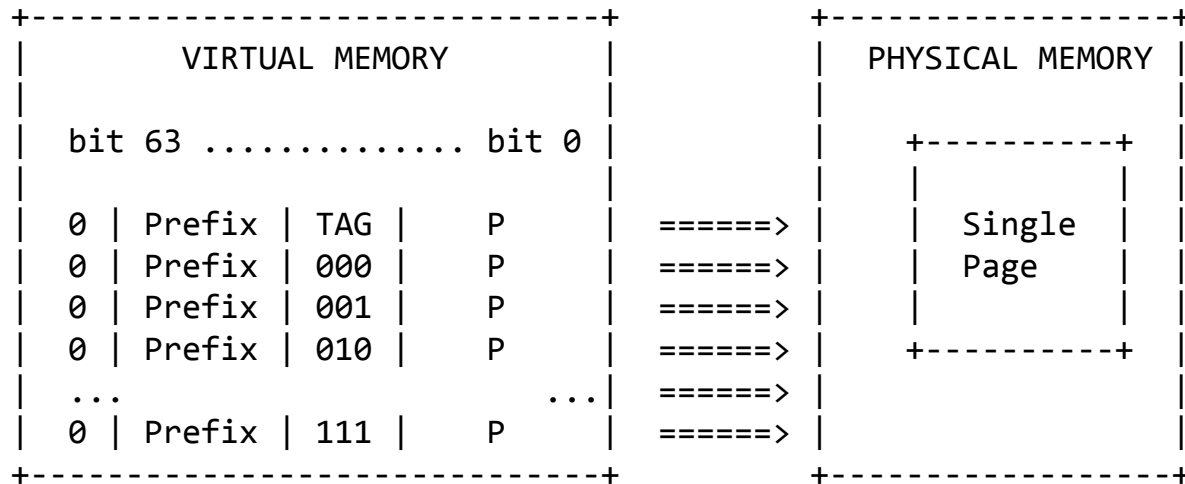
Hardware Memory Tagging (ARM MTE, SPARC ADI)

- Store small **tags** in pointers + memory; mismatch → fault
- Catches UAF by invalidating/retagging on free/reuse
- Low overhead in hardware; limited availability on x86-64
- Browsers/OSes can leverage MTE to raise attacker cost

Software Tagging (xTag and Memory Coloring)

xTag:

uses the processor's MMU to alias multiple virtual pages to the same physical page, embedding tag metadata into pointer-relevant bits without requiring special hardware.



xTag page-aliasing for software pointer tagging: changing the 4-bit metadata (e.g., 0000→0001) alters the virtual address, but all 2^4 aliases map to the same physical page so any tagged pointer still dereferences the same allocation.

Capability Architectures and Pointer Revocation

- **Design goal: address the root cause of UAF by eliminating dangling pointers at the hardware level**
- CHERI design
 - Implementation: CHERInvoke
 - provide fat pointer capabilities that include bounds and permissions
 - Cornucopia
 - architectural revocation system for CHERI heaps aiming for full temporal memory safety by sweeping memory for stale pointers and invalidating them in hardware

Fault Isolation and Compartmentalization

- **Design goal: isolating the effects of UAF errors to the process's sandbox**
 - Chrome's Site Isolation
 - Example: RCE cannot transform into UXSS
 - Out-of-Process JIT compilation
 - Firefox's RLBox
- microkernel or compartmentalization principles to isolate memory regions of different subsystems...



CASE STUDIES OF UAF EXPLOITS AND MITIGATIONS

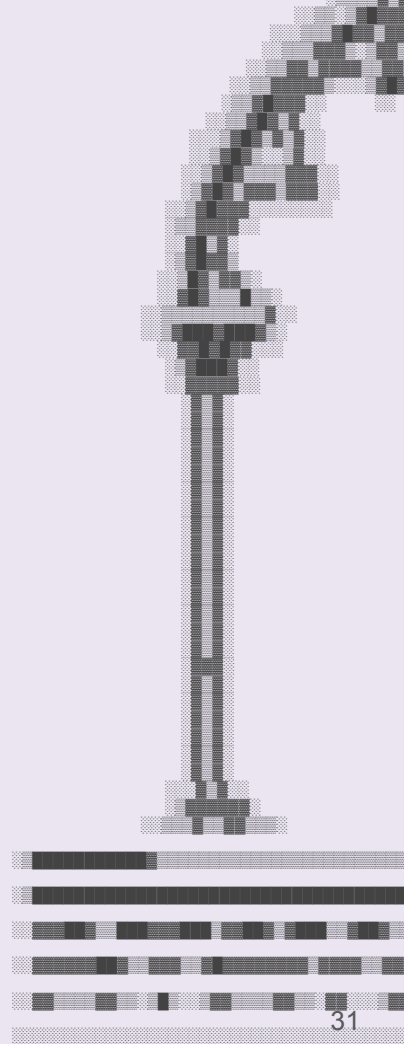
Case Study #1: mRuby Issue #4001 (CVE-2018-10199)

Problem: `mruby_io_initialize_copy` frees an I/O object and, on certain exceptions, exits without reassigning `DATA_PTR(copy)`, leaving a dangling pointer.

Exploitation: free I/O (A), place string (B) in its slot, then use the stale I/O pointer to overwrite B's critical fields, gaining out-of-bounds read/write via the corrupted string length.

Actual Fix: Upstream patch ("Fix use after free in File#initialize_copy") adjusted initialization/ordering to avoid the dangling pointer

Mitigations: Type isolation (e.g., PartitionAlloc/Cling); free-block canaries (FBC) and randomized in-block offsets (RIO) .



Case Study #2: PHP Session Deserializer UAF (CVE-2015-6835)

Problem: During session decoding, `php_var_unserialize()` returns a new **zval** that is freed by the caller while a reference to the same zval persists in `SESSION_VARS` → dangling pointer.

Exploitation: Heap placement tricks (re-runs/sprays) to force reuse of the freed slot so the stale pointer points into the attacker-controlled zval, enabling memory disclosure or RCE primitives.

Actual Fix: Patched in **PHP 5.4.45 / 5.5.29 / 5.6.13** by correcting the deserializer's handling of multiple `php_var_unserialize` calls (removing the UAF condition).

Mitigations: **FBC; RIO**; pure entropy allocators (Guarder/DieHarder) are weaker here; **context/type-based** allocators (**Cling/TypeAfterType**) can fail.

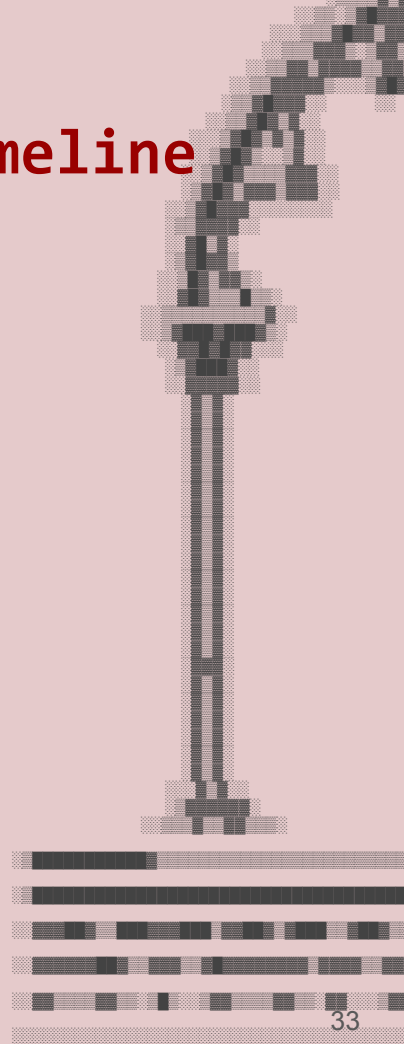
Case Study #3: Firefox Web Animations timeline UAF (CVE-2024-9680)

Problem: During `AnimationTimeline::Tick()`, re-entrancy via JS promise callbacks can cancel/remove an animation while the list is being iterated, leaving a dangling `Animation` pointer.

Exploitation: Exploited in the wild for content-process RCE, then chained with a Windows Task Scheduler EoP (CVE-2024-49039) to escape the sandbox.

Actual Fix: Mozilla's patch holds `RefPtr` to animations while ticking (stronger lifetime guarantees, preventing them from being freed during iteration), shipped in Firefox 131.0.2 / ESR 128.3.1 / ESR 115.16.1.

Mitigations: **Allocator hardening** (canaries, partitioned heaps); **fault isolation** (e.g., RLBox for 3rd-party libs / process isolation); **memory tagging**.



**DISCUSSION:
TRADE-OFFS AND
EFFECTIVENESS OF UAF
DEFENSES**

Trade-Offs: Guarantees vs Assumptions

- **Probabilistic** (ASLR, random reuse): cheap, but retry/leaks can win
- **Deterministic** (never-reuse pages, tagging): stronger, higher cost/coverage demands
- Defense-in-depth: combine partitioning + canaries + CFI + sandboxing

Cost and Practicality

- Many modern schemes keep runtime overhead to a few percents
- Memory overhead varies widely (quarantine, isolation)
- Integration complexity matters; production favors drop-in allocators

Mitigation x Properties Matrix

Mitigation	Coverage	Enforcement	CPU	Mem	Deployment	Notes
Guarder (USENIX’18)	Temporal	Probabilistic	~2–3%	—	Research	Entropy-based random slot selection to reduce targeted reuse.
SlimGuard (Middleware’19)	Temporal	Probabilistic (tunable r)	a few %	lower than DieHarder	Research	Space-efficient randomized reuse; tunable entropy.
S2malloc (2024)	Temporal	Hybrid (prob.+detection)	~2.8%	~27%	Research	FBC (write detection) + RIO (prefix misalignment).
Oscar (USENIX’17)	Temporal	Deterministic (never-reuse)	—	~60%	Research	Page-permission tricks to prevent slot reallocation.
FFmalloc (USENIX’21)	Temporal	Deterministic (batched never-reuse)	~2.3%	—	Research	Batching reduces runtime overhead at cost of memory.
PartitionAlloc (Chrome)	Both (spatial + partial temporal)	Deterministic	low	low/neutral	Production	Type/size partitioning, out-of-line metadata, guard pages.
MemGC (IE/Edge)	Temporal	Deterministic (quarantine)	amortized	transient increase	Production	Delayed frees; effectiveness may drop under OOM flush.
FreeGuard (CCS’17)	Temporal	Probabilistic (levels)	—	—	Research	Tunable “security levels”; Guarder-like randomized reuse.

[illegible]

CONCLUSIONS AND FUTURE WORK

Mitigation x Properties Matrix

Mitigation	Coverage	Enforcement	CPU	Mem	Deployment	Notes
DieHarder (CCS’10)	Both (probabilistic)	Probabilistic	a few %	up to ~57%	Research	Classic randomized allocator; influential baseline.
Cling (USENIX’10)	Temporal (type isolation)	Deterministic	~11%	—	Research	Per-type/context pools to block cross-type UAF reuse.
Type-after-Type (ACSAC’18)	Temporal (type-safe reuse)	Deterministic	~4%	—	Research	Reuse only within same type; call-site hash fallback.
Scudo (LLVM/Android)	Temporal	Probabilistic (quarantine)	—	—	Production	Hardened general-purpose allocator (Android 11+ for native code).
hardened_malloc (GrapheneOS)	Both (hardened checks)	Deterministic	—	—	Production	Strong checks; accepts higher perf/memory costs.
ZEUS / Protected heap allocation (DSC’18)	Temporal	Probabilistic	—	—	Research	RIO-like per-allocation prefix randomization (browser-focused).
Windows LFH (front-end heap)	Temporal (reuse randomness)	Probabilistic	—	—	Production	Bucketed small allocs; reduces determinism of reuse timing.

[illegible]

Key Takeaways

- **Layering works**
 - Allocator hardening + sandboxing + CFI increasingly turn UAFs into crash-only unless chained.
- **Deploy now (low friction):**
 - Partitioned/isolated heaps, quarantines, free-block canaries; enforce CFI; sandbox/RLBox high-risk libs.
- **Adopt when available: ARM MTE;**
 - evaluate **xTag/CHERI**; expand **Rust** for new components; use pointer-invalidation in hotspots.
- Practical allocators usually $\leq \sim 5\%$ **runtime**; memory costs vary (e.g., S2malloc ~27% RAM; Oscar ~60% RAM).

Deployment Decision Flow

- A. **Have hardware tags?** → Enable **MTE** + keep allocator hardening.
- B. **No hardware tags?** → Use **quarantine + canary + partitioned/isolated heaps** as baseline.
- C. **Threat model includes info-leaks or chains?** → Add **CFI** and stronger **process/compartment isolation**.

Open Research & Next Steps

- Hardware–software co-design for temporal safety (MTE)
- Better automatic pointer invalidation & analyses
- App-specific strategies (browser-aware randomization/tagging)
- Bringing strong checks to kernels without breaking performance

THANK YOU!
Q&A!?